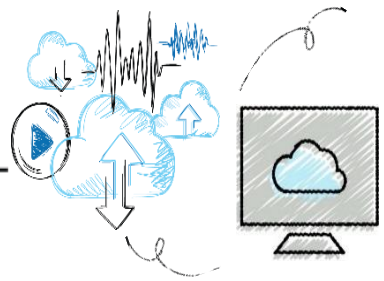
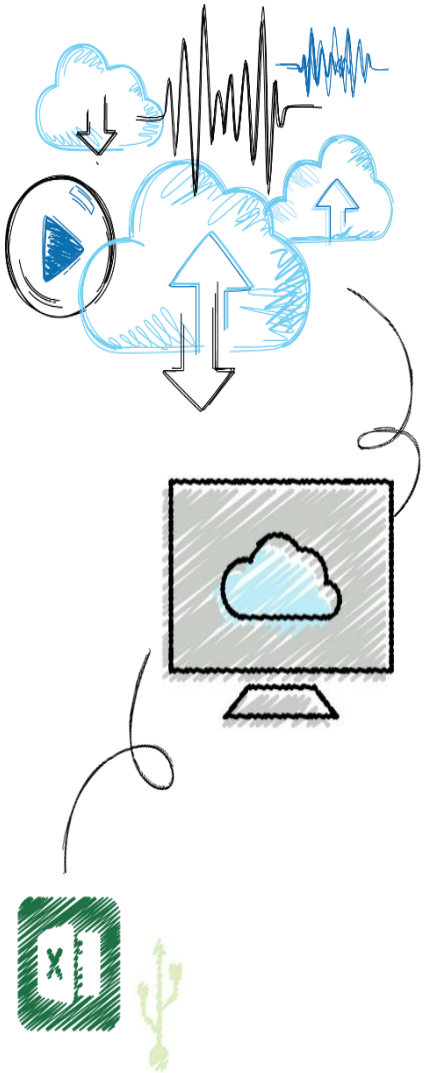
ICT703 Programming

Module 4 Topic 1

Strings





Objectives

1. Access individual characters in a string
2. Retrieve a substring from a string
3. Search for a substring in a string
4. Use string methods to manipulate strings

Programming Concepts

Four control flow

- Sequence
- Iteration
- Selection
- Abstraction – methods/classes

Data

The Structure of Strings

- An integer can't be factored into more primitive parts
- A string is a **data structure**
  - Data structure: Consists of smaller pieces of data
  - String's length: Number of characters it contains (0+)
- The **len** function returns the string's length, which is the number of characters it contains

```
len("Hi there!")
9
len("")
0
```

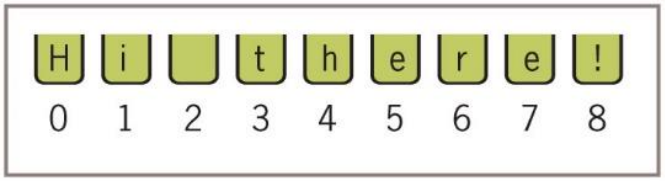


Figure 4-1 Characters and their positions in a string

Functions, Operators and Methods

- There are many ways to manipulate strings in python
- Functions
- Operators
- Methods

Python functions

- Some functions cannot be used with strings:
  - sum()
  - abs()

- Others can be used:
- len()
- min()
- max()

String operators

Operator	Description	Syntax	
+	Concatenates (joins) *string1* and *string2*	string1 + string2	a = "Tea " + "Leaf" print(a)
*	Repeats the string for as many times as specified by *x*	string * x	a = "Bee " * 3 print(a)
[]	Slice — Returns the character from the index provided at *x*.	string[x]	a = "Sea" print(a[1])
[:]	Range Slice — Returns the characters from the range provided at *x:y*.	string[x:y]	a = "Mushroom" print(a[4:8])
in	Membership — Returns True if *x* exists in the string. Can be multiple characters.	x in string	a = "Mushroom" print("m" in a) print("b" in a) print("shroo" in a)
not in	Membership — Returns True if *x* does *not* exist in the string. Can be multiple characters.	x not in string	a = "Mushroom" print("m" not in a) print("b" not in a) print("shroo" not in a)

The Subscript Operator

- The form of the **subscript operator** is:

  <a string>[<an integer expression>]

- Example:

```
name = "Alan Turing"
name[0]                    # Examine the first character
'A'
name[3]                    # Examine the fourth character
'n'
name[len(name)]
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
IndexError: string index out of range
name[len(name) - 1]        # Examine the last character
'g'
 name[-l]                  # Shorthand for the last character
'g'
name[-2]                   # Shorthand for next to last character
'n'
```

The Subscript Operator

- Subscript operator is useful when you want to use the positions as well as the characters in a string
  - Use a count-controlled loop

```
data = "Hi there! "
for index in range(len(data)):
        print(index, data[index])
```

```
0 H
1 i
2
3 t
4 h
5 e
6 r
7 e
8 !
```

Slicing for Substrings

- Python's subscript operator can be used to obtain a substring through a process called **slicing**
  - Place a colon (:) in the subscript; an integer value can appear on either side of the colon

```
name = "myfile.txt"          # The entire string
name[0:]
'myfile.txt'
name[0:1]                    # The first character
'm'
name[0:2]                    # The first two characters
'my'
name[:len(name)]             # The entire string
'myfile.txt'
name[–3:]                    # The last three characters
'txt'
name[2:6]                    # Drill to extract 'file'
'file'
```

Testing for a Substring with the in Operator

- When used with strings, the left operand of **in** is a target substring and the right operand is the string to be searched
  - Returns **True** if target string is somewhere in search string, or **False** otherwise
- This code segment traverses a list of filenames and prints just the filenames that have a .txt extension:

```
fileList = [“myfile.txt”, “myprogram.exe”, “yourfile.txt”]
for fileName in fileList:
        if “.txt” in fileName:
                print(fileName)
```

```
myfile.txt
yourfile.txt
```

String methods

- To further manipulate strings, python has in-built string methods

- All string methods return new values.
- They do not change the original string.

String Methods

- A method behaves like a function, but has a slightly different syntax
  - A method is always called with a given data value called an **object**

    <an object>.<method name>(<argument-1>,..., <argument-n>)

- Methods can expect arguments and return values
- A method knows about the internal state of the object with which it is called
- In Python, all data values are objects

String methods

capitalize()	Converts the first character to upper case
casefold()	Converts string into lower case
center()	Returns a centered string
count()	Returns the number of times a specified value occurs in a string
encode()	Returns an encoded version of the string
endswith()	Returns true if the string ends with the specified value
expandtabs()	Sets the tab size of the string
find()	Searches the string for a specified value and returns the position of where it was found
format()	Formats specified values in a string
isupper()	Returns True if all characters in the string are upper case
join()	Joins the elements of an iterable to the end of the string
ljust()	Returns a left justified version of the string
lower()	Converts a string into lower case
lstrip()	Returns a left trim version of the string
maketrans()	Returns a translation table to be used in translations
partition()	Returns a tuple where the string is parted into three parts
replace()	Returns a string where a specified value is replaced with a specified value
rfind()	Searches the string for a specified value and returns the last position of where it was found
rindex()	Searches the string for a specified value and returns the last position of where it was found
rjust()	Returns a right justified version of the string
rpartition()	Returns a tuple where the string is parted into three parts

index()	Searches the string for a specified value and returns the position of where it was found
isalnum()	Returns True if all characters in the string are alphanumeric
isalpha()	Returns True if all characters in the string are in the alphabet
isdecimal()	Returns True if all characters in the string are decimals
isdigit()	Returns True if all characters in the string are digits
isidentifier()	Returns True if the string is an identifier
islower()	Returns True if all characters in the string are lower case
isnumeric()	Returns True if all characters in the string are numeric
isprintable()	Returns True if all characters in the string are printable
isspace()	Returns True if all characters in the string are whitespaces
istitle()	Returns True if the string follows the rules of a title
rsplit()	Splits the string at the specified separator, and returns a list
rstrip()	Returns a right trim version of the string
split()	Splits the string at the specified separator, and returns a list
splitlines()	Splits the string at line breaks and returns a list
startswith()	Returns true if the string starts with the specified value
strip()	Returns a trimmed version of the string
swapcase()	Swaps cases, lower case becomes upper case and vice versa
title()	Converts the first character of each word to upper case
translate()	Returns a translated string
upper()	Converts a string into upper case
zfill()	Fills the string with a specified number of 0 values at the beginning

Commonly used methods

Method	True if
str.center(width)	Returns a copy of str centered within the given number of columns.
str.count(sub [, start [, end]])	Returns the number of non-overlapping occurrences of substring sub in str. Optional arguments start and end are interpreted as in slice notation.
str.join(sequence)	Returns a string that is the concatenation of the strings in the sequence. The separator between elements is str.
str.lower()	Returns a copy of str converted to lowercase.
str.upper()	Returns a copy of str converted to uppercase.
str.split([sep])	**Returns a list** of the words in str, using sep as the delimiter string. If sep is not specified, any whitespace string is a separator.
str.strip([aString])	Returns a copy of str with leading and trailing whitespace (tabs, spaces, newlines) removed. If aString is given, remove characters in aString instead.

Let's look at some of the Boolean methods

Method	True if
str.isalnum()	String consists of only alphanumeric characters (no symbols)
str.isalpha()	String consists of only alphabetic characters (no symbols)
str.islower()	String's alphabetic characters are all lower case
str.isnumeric()	String consists of only numeric characters
str.isspace()	String consists of only whitespace characters
str.istitle()	String is in title case
str.isupper()	String's alphabetic characters are all upper case

String Methods – split()

- Use split to count the words in a single sentence easy

```
sentence = input("Enter a sentence: ")
listOfWords = sentence.split()
print("There are", len(listOfWords), "words. ")
sum = 0
for word in listOfWords:
    sum += len(word)
print("The average word length is", sum / len(listOfWords))
```